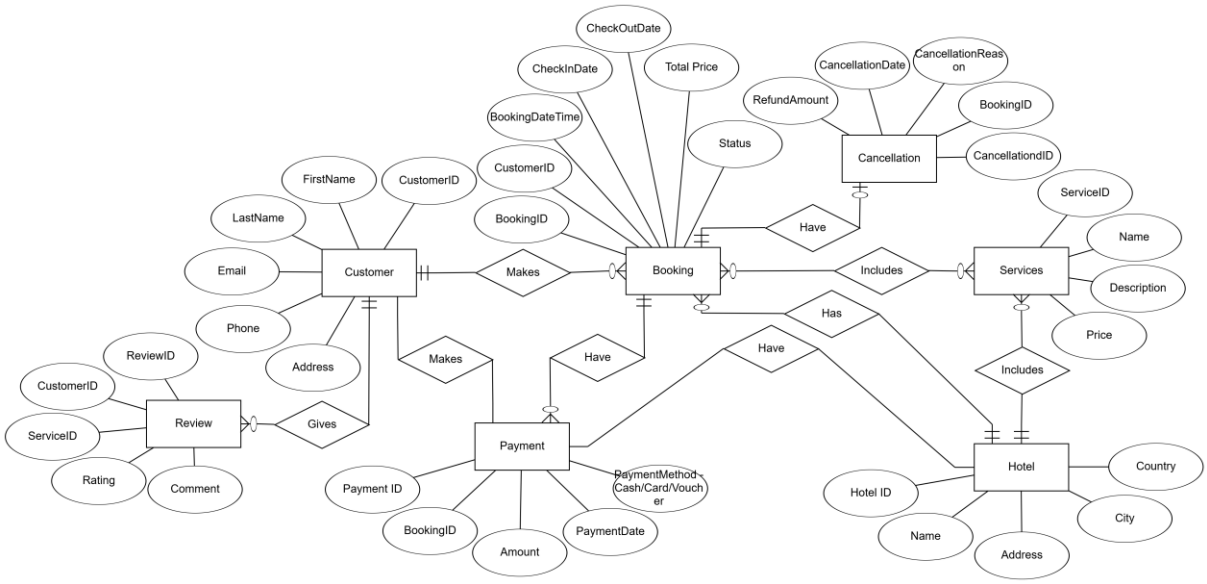
ZADANIE Z PREDMETU DATABÁZOVÉ SYSTÉMY

NÁZOV DOMÉNY: BOOKING.COM

MENO A PRIEZVISKO: DMYTRO ISAI
AK. ROK: 2024/2025

ENTITNO-RELAČNÝ DIAGRAM



OPIS ENTITNÝCH MNOŽÍN

1. Customer (Zákazník)
   Uchováva osobné údaje o zákazníkovi: meno, priezvisko, e-mail, telefónne číslo a adresu. Každý zákazník môže vytvoriť jednu alebo viac rezervácií (Booking).

2. Booking (Rezervácia)
   Obsahuje informácie o rezervácii: dátum a čas vytvorenia (BookingDateTime), termín príchodu/odchodu (CheckInDate, CheckOutDate), stav rezervácie (Status) a celkovú cenu (TotalPrice). Každá rezervácia je priradená jednému zákazníkovi (Customer) a jednému hotelu (Hotel).

3. Payment (Platba)
   Ukladá údaje o platbe za konkrétnu rezerváciu: spôsob platby (PaymentMethod), dátum platby (PaymentDate) a zaplatenú sumu (Amount). Platba je vždy naviazaná na jednu rezerváciu.

4. Services (Služby)
   Reprezentuje doplnkové služby (napr. raňajky, wellness a pod.), ktoré poskytuje konkrétny hotel (Hotel). Obsahuje názov služby (Name), opis (Description) a cenu (Price).

5. Cancellation (Zrušenie)
   Eviduje zrušenie rezervácie: dátum zrušenia (CancellationDate), dôvod (CancellationReason) a prípadnú vrátenú sumu (RefundAmount). Ak rezervácia nebola zrušená, v tejto entite pre ňu záznam neexistuje.

6. Hotel
   Uchováva údaje o ubytovacom zariadení: názov (Name), adresu (Address), mesto (City) a krajinu (Country). Hotel môže ponúkať viacero služieb (Services).

7. Review (Recenzia)

Umožňuje zákazníkovi ohodnotiť poskytnutú službu. Recenzia obsahuje hodnotenie (Rating) a komentár (Comment). Viaže sa na konkrétneho zákazníka (Customer) a konkrétnu službu (Services).

OPIS VZŤAHOV MEDZI ENTITNÝMI MNOŽINAMI

1. Customer – Booking
   „Makes“ znamená, že zákazník vytvára (robí) rezerváciu. Je to 1:N vzťah (jeden zákazník, viac rezervácií).

2. Booking – Payment
   „Have“ pri Booking a Payment poukazuje na to, že jedna rezervácia môže mať viac platieb. Tiež 1:N vzťah (v Payment je FK na Booking).

3. Booking – Cancellation
   1:1 (voliteľné) jedna rezervácia môže mať najviac jedno zrušenie, ale môže ho aj nemať.

4. Booking – Hotel
   Znamená, že rezervácia prebieha v konkrétnom hoteli. Z pohľadu Hotel ide o 1:N (jeden hotel môže mať viacero rezervácií).

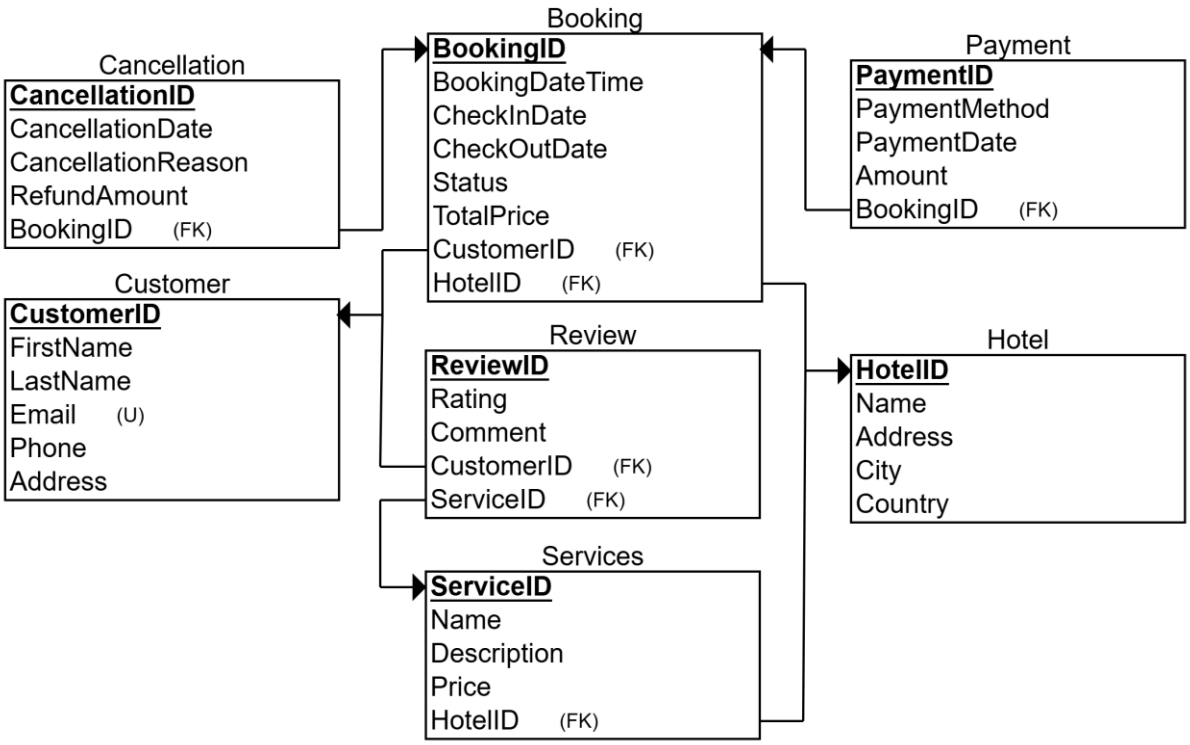
5. Booking – Services
   „Includes“ poukazuje na to, že rezervácia môže zahŕňať viaceré služby, pričom jedna služba môže byť použitá vo viacerých rezerváciách - M:N.

6. Hotel – Services
   „Includes“ znamená, že hotel môže ponúkať viacero služieb. 1:N (jeden hotel, viac služieb).

7. Customer – Review
   „Gives“ znamená, že zákazník môže napísať viac recenzií. 1:N (jeden zákazník, viac recenzií).

8. Services – Review
   1:N - Jedna služba môže mať viac recenzií. V Review je FK ServiceID.

RELAČNÝ MODEL



PRVÁ ODOVZDÁVKA - SQL SKRIPT NA VYTVORENIE DATABÁZY

```sql
CREATE TABLE Customer(
    CustomerID SERIAL PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Phone VARCHAR(20),
    Address VARCHAR(255)
);
```

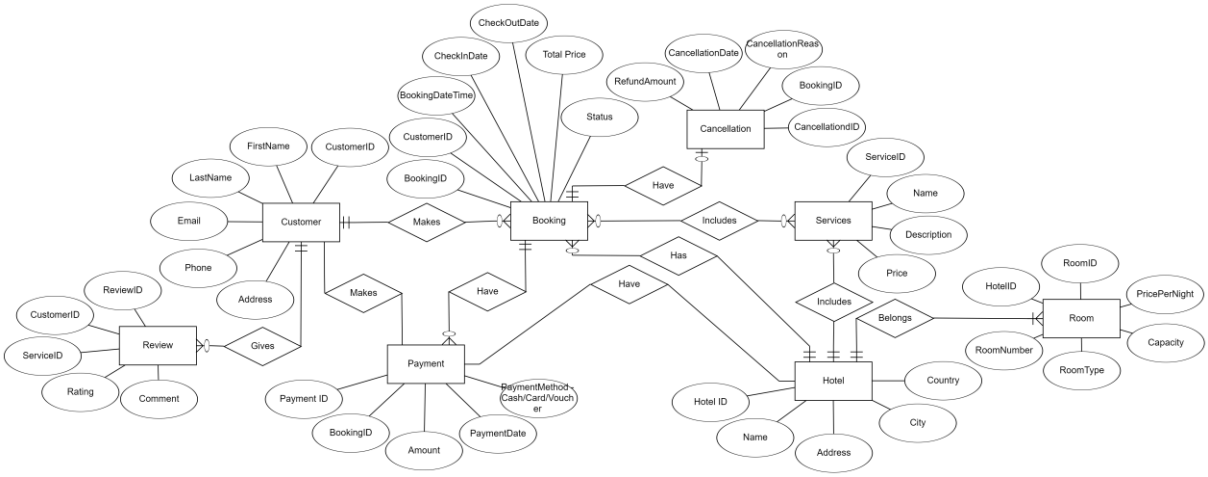
```sql
CREATE TABLE Hotel(
    HotelID SERIAL PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Address VARCHAR(255),
    City VARCHAR(50),
    Country VARCHAR(50)
);

CREATE TABLE Booking(
    BookingID SERIAL PRIMARY KEY,
    CustomerID INT NOT NULL,
    HotelID INT NOT NULL,
    BookingDateTime TIMESTAMP NOT NULL,
    CheckInDate DATE,
    CheckOutDate DATE,
    Status VARCHAR(20) NOT NULL,
    TotalPrice NUMERIC(10,2),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (HotelID) REFERENCES Hotel(HotelID)
);


CREATE TABLE Payment(
    PaymentID SERIAL PRIMARY KEY,
    BookingID INT NOT NULL,
    PaymentMethod VARCHAR(50) NOT NULL,
    PaymentDate TIMESTAMP NOT NULL,
    Amount NUMERIC(10,2) NOT NULL,
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)
);

CREATE TABLE Cancellation(
    CancellationID SERIAL PRIMARY KEY,
    BookingID INT NOT NULL,
    CancellationDate TIMESTAMP NOT NULL,
    CancellationReason VARCHAR(255),
    RefundAmount NUMERIC(10,2),
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID)
);

CREATE TABLE Services(
    ServiceID SERIAL PRIMARY KEY,
    HotelID INT NOT NULL,
    Name VARCHAR(100) NOT NULL,
    Description VARCHAR(255),
    Price NUMERIC(10,2) NOT NULL,
    FOREIGN KEY (HotelID) REFERENCES Hotel(HotelID)
);
```

```sql
CREATE TABLE Review(
    ReviewID SERIAL PRIMARY KEY,
    CustomerID INT NOT NULL,
    ServiceID INT NOT NULL,
    Rating INT CHECK (Rating BETWEEN 1 AND 5),
    Comment VARCHAR(255),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (ServiceID) REFERENCES Services(ServiceID)
);
```

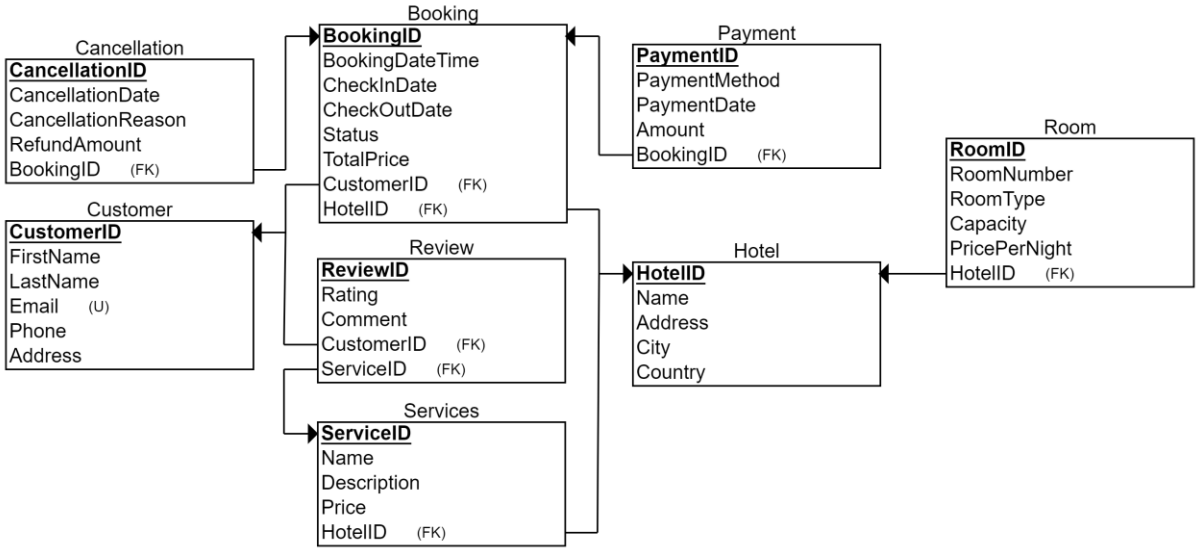
DRUHÁ ODOVZDÁVKA

Opravil a pridal:

ER-diagram



RS-diagram



Room (Izba) - Entita Room reprezentuje jednotlivé izby, ktoré sú súčasťou hotela. Každá izba má svoje špecifické atribúty, ktoré ju odlišujú – číslo izby, typ izby, kapacitu a cenu za noc.

Hotel – Room - Jeden hotel obsahuje viac izieb (1:N), pričom každá izba nevyhnutne patrí práve jednému hotelu (N:1).

Code:

```
CREATE TABLE Room(
    RoomID SERIAL PRIMARY KEY,
    HotelID INT NOT NULL,
    RoomNumber VARCHAR(20) NOT NULL,
    RoomType VARCHAR(50) NOT NULL,
    Capacity INT NOT NULL,
    PricePerNight NUMERIC(10,2) NOT NULL,
    FOREIGN KEY (HotelID) REFERENCES Hotel(HotelID)
);

CREATE TABLE Booking_Room(
    BookingID INT NOT NULL,
    RoomID INT NOT NULL,
    PRIMARY KEY (BookingID, RoomID),
    FOREIGN KEY (BookingID) REFERENCES Booking(BookingID),
    FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
);
```

SKRIPT NA NAPLNENIE DATABÁZY (KAŽDÁ TABUĽKA ASPOŇ 5 ZÁZNAMOV)

```sql
INSERT INTO Customer (FirstName, LastName, Email, Phone, Address)
VALUES
('Peter', 'Novak', 'peter.novak@gmail.com', '+421-905-123456', 'Hlavná 10,
Bratislava, SK'),
('Mária', 'Horváthová', 'maria.horvathova@gmail.com', '+421-908-654321', 'Námestie
SNP 5, Košice, SK'),
('Alice', 'Brown', 'alice.brown@outlook.com', '+1-202-555-0177', '789 Oak Ave'),
('Bob', 'Johnson', 'bob.johnson@gmail.com', '+1-202-555-0155', '321 Pine Rd'),
('Carol', 'Williams', 'carol.williams@gmail.com', '+1-202-555-0111', '654 Maple
Dr');

INSERT INTO Hotel (Name, Address, City, Country)
VALUES
('Slnečný Hotel', 'Hlavná 1', 'Bratislava', 'SK'),
('Moonlight Resort', 'Island Road 22', 'Honolulu', 'USA'),
('Crystal', 'Slovenskej Jednoty 1776', 'Košice', 'SK'),
('Royal Inn', 'King Street 10', 'London', 'UK'),
('Mountain View Lodge', 'Highland Rd 50', 'Denver', 'USA');


INSERT INTO Room (HotelID, RoomNumber, RoomType, Capacity, PricePerNight)
VALUES
(1,'101','Standard',2,100.00),
(1,'102','Deluxe',3,150.00),
(2,'201','Suite',4,200.00),
(3,'301','Standard',2,120.00),
(4,'401','Executive',2,180.00),
(5,'501','Standard',2,110.00),
(5,'502','Family',4,160.00);

INSERT INTO Booking (CustomerID, HotelID, BookingDateTime, CheckInDate,
CheckOutDate, Status, TotalPrice)
VALUES
(1,1,NOW(),'2025-03-10','2025-03-15','Confirmed',500.00),
(2,2,NOW(),'2025-06-01','2025-06-07','Pending',800.00),
(3,3,NOW(),'2025-05-05','2025-05-10','Confirmed',600.00),
(4,4,NOW(),'2025-07-20','2025-07-25','Cancelled',700.00),
(5,5,NOW(),'2025-08-15','2025-08-20','Confirmed',650.00);

INSERT INTO Booking_Room (BookingID, RoomID)
VALUES
(1,1),
(1,2),
(2,3),
(3,4),
(4,5),
(5,6),
(5,7);
```

```sql
INSERT INTO Payment (BookingID, PaymentMethod, PaymentDate, Amount)
VALUES
(1,'Credit Card',NOW(),500.00),
(2,'PayPal',NOW(),200.00),
(2,'Credit Card',NOW(),600.00),
(3,'Debit Card',NOW(),600.00),
(5,'Credit Card',NOW(),650.00);

INSERT INTO Cancellation (BookingID, CancellationDate, CancellationReason,
RefundAmount)
VALUES
(4,NOW(),'Customer changed plans',350.00),
(2,NOW(),'Booking cancelled by customer',100.00),
(5,NOW(),'Overbooking issue',150.00),
(3,NOW(),'Mistake in booking',50.00),
(1,NOW(),'Hotel maintenance issue',200.00);

INSERT INTO Services (HotelID, Name, Description, Price)
VALUES
(1,'Breakfast','Buffet breakfast included',20.00),
(2,'Spa Access','Full day spa pass',50.00),
(3,'Gym Access','24/7 gym access',15.00),
(4,'Airport Transfer','Pickup and dropoff service',40.00),
(5,'Room Service','24-hour room service',30.00);

INSERT INTO Review (CustomerID, ServiceID, Rating, Comment)
VALUES
(1,1,5,'Excellent breakfast!'),
(2,2,4,'Great spa experience.'),
(3,3,3,'Gym was average.'),
(4,4,2,'Airport transfer was late.'),
(5,5,4,'Room service was prompt and friendly.');
```

2 POHĽADY S JEDNODUCHÝM NETRIVIÁLNYM SELECTOM NAD JEDNOU TABUĽKOU

```sql
CREATE VIEW View_Customers_Gmail AS
SELECT CustomerID, FirstName, LastName, Email
FROM Customer
WHERE Email LIKE '%gmail.com%';

CREATE VIEW View_Hotels_USA AS
SELECT HotelID, Name, City, Country
FROM Hotel
WHERE Country = 'USA';
```

3 POHĽADY SO SPÁJANÍM TABULIEK
(1X SPOJENIE ASPOŇ 2 TABULIEK, 1X SPOJENIE ASPOŇ 3 TABULIEK, 1X OUTER JOIN)

```sql
CREATE VIEW View_Booking_Customers AS
SELECT b.BookingID, c.FirstName, c.LastName, b.BookingDateTime, b.Status,
b.TotalPrice
FROM Booking b
JOIN Customer c ON b.CustomerID = c.CustomerID;

CREATE VIEW View_Full_Booking_Info AS
SELECT b.BookingID, c.FirstName, c.LastName, h.Name AS HotelName, h.City,
b.BookingDateTime, b.Status, b.TotalPrice
FROM Booking b
JOIN Customer c ON b.CustomerID = c.CustomerID
JOIN Hotel h ON b.HotelID = h.HotelID;

CREATE VIEW View_Booking_Payment AS
SELECT b.BookingID, b.Status, p.PaymentMethod, p.Amount
FROM Booking b
LEFT JOIN Payment p ON b.BookingID = p.BookingID;
```

2 POHĽADY S POUŽITÍM AGREGAČNÝCH FUNKCIÍ A/ALEBO ZOSKUPENIA

```sql
CREATE VIEW View_Avg_Booking_By_Hotel AS
SELECT h.HotelID, h.Name AS HotelName, AVG(b.TotalPrice) AS AvgPrice,
COUNT(b.BookingID) AS NumBookings
FROM Booking b
JOIN Hotel h ON b.HotelID = h.HotelID
GROUP BY h.HotelID, h.Name;

CREATE VIEW View_Service_Review_Stats AS
SELECT s.ServiceID, s.Name AS ServiceName, AVG(r.Rating) AS AvgRating,
COUNT(r.ReviewID) AS NumReviews
FROM Review r
JOIN Services s ON r.ServiceID = s.ServiceID
GROUP BY s.ServiceID, s.Name;
```

TRETIA ODOVZDÁVKA

1 POHĽAD S POUŽITÍM MNOŽINOVÝCH OPERÁCIÍ

```
CREATE VIEW View_Email_Providers AS
SELECT CustomerID, FirstName, Email FROM Customer WHERE Email LIKE '%gmail.com%'
UNION
SELECT CustomerID, FirstName, Email FROM Customer WHERE Email LIKE
'%outlook.com%';
```

2 POHĽADY S POUŽITÍM VNORENÝCH PODDOPYTOV

```
CREATE VIEW View_Customers_NoBooking AS
SELECT CustomerID, FirstName, LastName
FROM Customer c WHERE NOT EXISTS(SELECT 1 FROM Booking b WHERE
b.CustomerID=c.CustomerID);

CREATE VIEW View_Top_Hotels_By_Service AS
SELECT DISTINCT h.HotelID, h.Name
FROM Hotel h
WHERE h.HotelID IN(
    SELECT s.HotelID
    FROM Services s JOIN Review r ON s.ServiceID=r.ServiceID
    GROUP BY s.HotelID
    HAVING AVG(r.Rating)>=4
 );
```

SKRIPT NA VYTVORENIE TRIGGERU/TRIGGEROV, KTORÉ BUDÚ IMPLEMENTOVAŤ AUTOINKREMENTÁCIU UMELÝCH KĽÚČOV

```
CREATE OR REPLACE FUNCTION set_id()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_ARGV[0] = 'customer' THEN IF NEW.CustomerID IS NULL THEN NEW.CustomerID
:= nextval('seq_customer'); END IF;
    ELSIF TG_ARGV[0] = 'hotel' THEN IF NEW.HotelID IS NULL THEN NEW.HotelID :=
nextval('seq_hotel'); END IF;
    ELSIF TG_ARGV[0] = 'room' THEN IF NEW.RoomID IS NULL THEN NEW.RoomID :=
nextval('seq_room'); END IF;
    ELSIF TG_ARGV[0] = 'booking' THEN IF NEW.BookingID IS NULL THEN NEW.BookingID
:= nextval('seq_booking'); END IF;
    ELSIF TG_ARGV[0] = 'payment' THEN IF NEW.PaymentID IS NULL THEN NEW.PaymentID
:= nextval('seq_payment'); END IF;
    ELSIF TG_ARGV[0] = 'cancellation' THEN IF NEW.CancellationID IS NULL THEN
NEW.CancellationID := nextval('seq_cancellation'); END IF;
    ELSIF TG_ARGV[0] = 'services' THEN IF NEW.ServiceID IS NULL THEN NEW.ServiceID
:= nextval('seq_services'); END IF;
    ELSIF TG_ARGV[0] = 'review' THEN IF NEW.ReviewID IS NULL THEN NEW.ReviewID :=
nextval('seq_review'); END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_customer_ai BEFORE INSERT ON Customer FOR EACH ROW EXECUTE
FUNCTION set_id('customer');
CREATE TRIGGER trg_hotel_ai BEFORE INSERT ON Hotel FOR EACH ROW EXECUTE FUNCTION
set_id('hotel');
CREATE TRIGGER trg_room_ai BEFORE INSERT ON Room FOR EACH ROW EXECUTE FUNCTION
set_id('room');
CREATE TRIGGER trg_booking_ai BEFORE INSERT ON Booking FOR EACH ROW EXECUTE
FUNCTION set_id('booking');
CREATE TRIGGER trg_payment_ai BEFORE INSERT ON Payment FOR EACH ROW EXECUTE
FUNCTION set_id('payment');
CREATE TRIGGER trg_cancel_ai BEFORE INSERT ON Cancellation FOR EACH ROW EXECUTE
FUNCTION set_id('cancellation');
CREATE TRIGGER trg_services_ai BEFORE INSERT ON Services FOR EACH ROW EXECUTE
FUNCTION set_id('services');
CREATE TRIGGER trg_review_ai BEFORE INSERT ON Review FOR EACH ROW EXECUTE FUNCTION
set_id('review');
```

SKRIPT NA ASPOŇ DVA ZMYSLUPLNÉ TRIGGRE
(OKREM TRIGGEROV NA AUTOINKREMENTÁCIU)

```sql
CREATE OR REPLACE FUNCTION trg_set_booking_cancelled()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE Booking SET Status='Cancelled' WHERE BookingID=NEW.BookingID;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_cancel_booking
AFTER INSERT ON Cancellation
FOR EACH ROW EXECUTE FUNCTION trg_set_booking_cancelled();

CREATE OR REPLACE FUNCTION trg_update_payment_on_view()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE Payment
    SET PaymentMethod=NEW.PaymentMethod, Amount =NEW.Amount
    WHERE PaymentID=OLD.PaymentID;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_upd_payment_view
INSTEAD OF UPDATE ON View_Booking_Payment
FOR EACH ROW EXECUTE FUNCTION trg_update_payment_on_view();
```

SKRIPT NA ASPOŇ JEDNU STOROVANÚ PROCEDÚRU

```sql
CREATE OR REPLACE PROCEDURE sp_cancel_old_bookings(p_before_date DATE)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO
Cancellation(BookingID,CancellationDate,CancellationReason,RefundAmount)
    SELECT BookingID,NOW(),'Auto-cancel stale booking',TotalPrice*0.5
    FROM Booking
    WHERE CheckInDate<p_before_date AND Status<>'Cancelled';
END;
$$;
```

SKRIPT NA ASPOŇ JEDNU FUNKCIU

```sql
CREATE OR REPLACE FUNCTION get_available_rooms(p_hotel_id INT, p_checkin DATE,
p_checkout DATE)
```

```
    RETURNS TABLE(RoomID INT, RoomNumber VARCHAR, RoomType VARCHAR, Capacity INT,
PricePerNight NUMERIC) AS $$
BEGIN
    RETURN QUERY
    SELECT r.RoomID, r.RoomNumber, r.RoomType, r.Capacity, r.PricePerNight
    FROM Room r
    WHERE r.HotelID=p_hotel_id
        AND r.RoomID NOT IN(
        SELECT br.RoomID FROM Booking_Room br
        JOIN Booking b ON br.BookingID=b.BookingID
        WHERE NOT(b.CheckOutDate<=p_checkin OR b.CheckInDate>=p_checkout)
        );
END;
$$ LANGUAGE plpgsql;
```